Mark	

Team name	*A10*
Homework number	*HOMEWORK 06*
Due date	30/10/2026

Contribution	NO	Partial	Full
Carbone Alessandro			*X*
Casareale Alberto			*X*
Colombi Federico			*X*
Lorusso Luca			*X*
Martorano Gabriele			*X*
Notes: Everyone contributed to understanding the logic and implementing the code			

Project name	6A - ADC single acquisition (interrupt)		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			X

Explanation:

Part 1: Hardware Configuration and Routing

To achieve the goal of acquiring the potentiometer voltage every second and sending it to a remote terminal, we first consulted the “Hands-on Lab Schematics.pdf”, verifying that the potentiometer is physically connected to pin PA1. We configured the system clock without enabling its related interrupts and set the ADC to operate in interrupt mode. To ensure exact and periodic sampling without using blocking delay functions, Timer 2 (TIM2) was configured to generate an "Update" event, while ADC1 was set to interrupt mode, using Timer 2 itself as the external trigger source to start the conversion (ADC_EXTERNALTRIGCONV_T2_TRGO). Additionally, we enabled the USART peripheral for reception (RX) with interrupts to support communication.

Part 2: Code Generation and Verification

After generating the project, we verified the basic settings within the main.c file. Timer 2 was initialized with a prescaler of 8400-1 and a period (ARR) of 10000-1. Considering the base frequency, this configuration ensures that the timer generates an "Update Event" exactly every second. In the main function, immediately before the infinite loop, the system is activated by starting the ADC in interrupt mode using the HAL_ADC_Start_IT(&hadc1) function and starting the Timer 2 count with HAL_TIM_Base_Start(&htim2).

Part 3: Functional Logic Implementation

The entire functional logic is handled in the background, leaving the while(1) loop completely empty. Every second, TIM2 automatically generates the trigger that starts the single ADC conversion. Upon completion

of the conversion, the system triggers the HAL_ADC_ConvCpltCallback interrupt routine. Within this callback, the code reads the raw digital value via HAL_ADC_GetValue(&hadc1) and calculates the actual voltage in millivolts by applying the formula (adc_val * 3300) / 4095. Finally, using the snprintf function, the values are inserted into a formatted string, which is immediately transmitted to the terminal via HAL_UART_Transmit.

Professor comments:

Project name	6B - ADC triggered by a timer, shown on the LCD		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			X

Explanation:

Part 1: Hardware Configuration and Routing

To acquire the potentiometer voltage at a regular interval of 1 Hz and display it numerically and graphically on the LCD display, we carefully configured the necessary peripherals. Consulting the “Hands-on Lab Schematics.pdf”, we verified that the potentiometer is physically connected to the analog pin PA1. Subsequently, we configured the dedicated GPIO pins (PA4, PB1, PB2, PB12-15) as outputs to drive the LCD module. The system clock was configured without enabling its related interrupts. To handle the sampling in hardware without blocking the CPU, Timer 2 (TIM2) was configured to generate an "Update" event, while ADC1 was set to interrupt mode, using Timer 2 itself as the external trigger source to start the conversion (ADC_EXTERNALTRIGCONV_T2_TRGO).

Part 2: Code Generation and Verification

At the development environment level, we integrated the PMDB16_LCD.c and PMDB16_LCD.h files into the project and updated the Makefile (adding the relevant directories in target_include_directories and target_sources). Additionally, the -u _printf_float flag was added to target_link_options to enable floating-point calculation support for the snprintf function. In the main_2.c file, we verified the TIM2 parameters: by setting the prescaler to 8400-1 and the period to 10000-1, we obtain a trigger frequency of exactly 1 Hz. In main(), immediately before the infinite loop, the system initializes the display, turns on its backlight, clears the screen, and finally activate the system by starting the timer count with HAL_TIM_Base_Start(&htim2) and the ADC in interrupt mode with HAL_ADC_Start_IT(&hadc1).

Part 3: Functional Logic Implementation

The entire functional logic of the system is handled in the background, leaving the while(1) loop completely empty. Every second, TIM2 automatically triggers an ADC conversion. Upon completion, the system calls the HAL_ADC_ConvCpltCallback. Inside it, the raw 12-bit value is read via HAL_ADC_GetValue and the actual voltage is calculated using the expression (adc_val * 3.3 / 4095). The voltage value is formatted into a msg string with three decimal places ("Voltage: %.3f V") and printed on the upper line of the display using

lcd_println(msg, 0). Finally, to meet the requirement for graphical display, the bottom line is updated using the lcd_drawBar function; the passed argument is the result of (adc_val * 80 / 4095), which linearly scales the ADC value (0-4095) based on the number of available columns/segments (80) for the screen display.

Professor comments: